Instituto Federal do Norte de Minas Gerais - Campus Salinas
Bacharelado em Sistemas de Informação

Sistema Web para Agendamento de Espaços Físicos no IFNMG – Salinas

Aluno: Murilo Santos Gonçalves

Orientador: Arthur Farias Porto

Coorientadora: Jéssica Glória

21 de outubro de 2026

1 INTRODUÇÃO

A organização dos espaços físicos num campus universitário pode aparentar simplicidade, mas revela-se complexa na prática. Quando laboratórios e salas especiais são compartilhados por vários docentes sem um sistema de agendamento adequado, surgem conflitos de horário e a tarefa de controlar quem usa o quê e quando recai sobre servidores que já têm outras atribuições.

No IFNMG – Campus Salinas, esse problema tem contorno específico. O sistema utilizado para agendamentos exige que todos os horários já ocupados sejam cadastrados manualmente antes de qualquer reserva, um processo que precisa ser refeito a cada troca de semestre, uma vez que as atividades dos espaços variam significativamente ao longo do período letivo. Além do retrabalho contínuo, os docentes não têm autonomia para consultar disponibilidade ou solicitar reservas sem intermediação, e a instituição não dispõe de registro consolidado sobre a utilização dos laboratórios ao longo do tempo. Esse cenário não é exclusivo do Campus Salinas: estudo de Cruz e Barroso (2023) no IFNMG Campus Almenara identificou a ausência de planejamento permanente e de mecanismos para medir a utilização dos laboratórios.

Este trabalho propõe o desenvolvimento de um sistema web para o gerenciamento de reservas de espaços físicos no IFNMG – Campus Salinas. A proposta visa permitir que docentes consultem a disponibilidade e realizem solicitações de reserva de forma autônoma, enquanto o servidor responsável pelos agendamentos acessa ferramentas de aprovação, recebe notificações e gera relatórios sobre o uso dos espaços.

Antes de optar pelo desenvolvimento de uma solução própria, foram investigadas alternativas já disponíveis no mercado e na comunidade de software livre, incluindo plataformas comerciais de reserva de espaços, como o Skedda (Skedda, 2025) e o Microsoft Bookings (Microsoft, 2025), além de sistemas acadêmicos desenvolvidos por outras instituições de ensino (CRUZ; BARROSO, 2023; NASCIMENTO; RIOS; BALUZ, 2020). Essa investigação evidenciou que nenhuma dessas soluções atende ao requisito central apontado pela Direção de Ensino Técnico do campus: fazer com que a disponibilidade dos espaços reflita automaticamente a grade de horários institucional.

A necessidade de uma solução específica foi expressa pela Diretora de Ensino Técnico do IFNMG – Campus Salinas, setor responsável pela gestão dos agendamentos, que destacou[1]:

> Precisamos de um software que forneça um mapa de utilização dos espaços do campus e permita que horários livres dos espaços sejam agendados pelos docentes, aprovados pelos administradores e visualizados por todos.

[1] Informação verbal fornecida pela Diretora de Ensino Técnico do IFNMG – Campus Salinas, em reuniões remotas realizadas entre julho de 2025 e setembro de 2026.

Espera-se que o sistema contribua para a otimização do uso dos espaços físicos, a redução de conflitos de agendamento e a melhoria na gestão institucional do campus.

1.1 OBJETIVO GERAL

Desenvolver um sistema web para o gerenciamento de reservas de espaços físicos no IFNMG – Campus Salinas.

1.2 OBJETIVOS ESPECÍFICOS

- Levantar o processo atual de agendamento e identificar as necessidades dos perfis de usuários envolvidos;
- Especificar os requisitos funcionais e não funcionais do sistema;
- Implementar o sistema web com as tecnologias e a arquitetura definidas a partir dos requisitos especificados;
- Disponibilizar ao administrador ferramentas para a gestão dos agendamentos e o acompanhamento do uso dos espaços;
- Verificar o funcionamento do sistema por meio de testes automatizados.

1.3 ESTRUTURA DO TRABALHO

Este trabalho está estruturado da seguinte forma: no Capítulo 2 é apresentada a revisão bibliográfica, que abrange o contexto institucional, os trabalhos relacionados e a fundamentação tecnológica; no Capítulo 3, a metodologia utilizada na condução da pesquisa; no Capítulo 4, o cronograma de execução das etapas previstas; e, por fim, as referências bibliográficas consultadas.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta o referencial teórico que embasa o problema e a solução propostos. A primeira parte situa o leitor no contexto institucional em que o problema ocorre: o IFNMG – Campus Salinas e a forma como instituições de ensino lidam com a gestão de espaços físicos. A segunda parte apresenta os trabalhos relacionados, descrevendo iniciativas semelhantes já documentadas na literatura e identificando em que pontos o presente trabalho se distingue delas. A terceira parte reúne os conceitos técnicos que fundamentam o desenvolvimento do sistema web proposto, cobrindo engenharia de software, modelagem com UML, banco de dados, controle de acesso, arquitetura web, usabilidade e testes.

2.1 CONTEXTO INSTITUCIONAL

Esta seção descreve o IFNMG – Campus Salinas e o funcionamento dos seus espaços físicos e apresenta o que a literatura registra sobre gestão de espaços em ambientes acadêmicos.

2.1.1 O IFNMG E O CAMPUS SALINAS

O Instituto Federal de Educação, Ciência e Tecnologia do Norte de Minas Gerais (IFNMG) é uma autarquia federal criada pela Lei n.º 11.892, de 29 de dezembro de 2008, que reorganizou a Rede Federal de Educação Profissional, Científica e Tecnológica no Brasil. Conforme seu Plano de Desenvolvimento Institucional, o IFNMG tem por missão promover educação profissional, científica e tecnológica por meio do ensino, da pesquisa e da extensão, com vistas à formação humana integral e ao desenvolvimento regional (Instituto Federal do Norte de Minas Gerais, 2024).

O Campus Salinas integra a rede do IFNMG e oferta cursos em diferentes níveis e modalidades, entre eles o Bacharelado em Sistemas de Informação, no âmbito do qual este trabalho está inserido. Para o funcionamento dessas atividades acadêmicas, o campus dispõe de laboratórios de informática e outras salas especiais cujo uso é compartilhado por diferentes turmas e docentes ao longo do período letivo, o que exige mecanismos de organização e controle sobre a ocupação desses espaços.

A natureza dos institutos federais agrava esse desafio. Ao contrário de instituições que ofertam um único tipo de curso, os IFs atendem simultaneamente a turmas de educação técnica integrada ao ensino médio, cursos técnicos subsequentes e cursos de nível superior, todos com horários distintos e, em muitos casos, disputando os mesmos laboratórios. Um laboratório de informática, por exemplo, pode ser utilizado em horários diferentes para aulas de disciplinas técnicas, para disciplinas do bacharelado e para atividades de extensão, o que torna a gestão manual da disponibilidade um trabalho de complexidade operacional (Instituto Federal do Norte de Minas Gerais, 2024).

2.1.2 A GESTÃO DE ESPAÇOS EM INSTITUIÇÕES DE ENSINO

A gestão dos espaços físicos em instituições de ensino inclui o planejamento do uso, o controle de disponibilidade, o registro de ocupações e o acompanhamento de como os recursos são efetivamente utilizados. Quando esses mecanismos não existem ou são precários, o resultado é a subutilização dos espaços e a dificuldade de responder a demandas concretas dos usuários.

Esse cenário se repete em outras instituições públicas de ensino, como mostram os trabalhos de Cruz e Barroso (2023) e Nascimento, Rios e Baluz (2020), discutidos na Seção 2.2.

2.2 TRABALHOS RELACIONADOS

Esta seção apresenta trabalhos e soluções que trataram de problemas semelhantes ao abordado nesta pesquisa, seja pelo diagnóstico da gestão de espaços em instituições federais de ensino, seja pelo desenvolvimento de sistemas de reserva em ambientes acadêmicos, seja pela oferta de ferramentas de mercado para a reserva de espaços. A análise desses trabalhos e soluções permite situar a presente proposta em relação ao que já foi produzido e identificar os aspectos que a diferenciam.

2.2.1 SOLUÇÕES EXISTENTES DE RESERVA DE ESPAÇOS

Além dos trabalhos acadêmicos, foram analisadas soluções de software já disponíveis para a reserva de espaços, com o objetivo de verificar se alguma delas resolveria o problema do Campus Salinas sem a necessidade de desenvolvimento de um sistema específico. A análise concentrou-se em duas plataformas comerciais representativas: o Skedda e o Microsoft Bookings.

O Skedda é uma plataforma web dedicada à reserva de espaços físicos, como salas, escritórios e laboratórios, que oferece autoatendimento aos usuários, definição de regras de reserva e visualização de disponibilidade em formato de agenda (Skedda, 2025). O Microsoft Bookings, integrante do pacote Microsoft 365, é voltado ao agendamento de compromissos e ao gerenciamento de calendários, com reserva de recursos por meio de caixas de correio associadas a salas (Microsoft, 2025). Ambas as ferramentas cobrem funcionalidades que também são esperadas do sistema proposto, como a consulta de disponibilidade, a solicitação de reserva pelo próprio usuário e a notificação automática das partes envolvidas, o que demonstra a maturidade desse tipo de aplicação no mercado.

Nenhuma das duas, contudo, resolve o problema central do campus: fazer a disponibilidade refletir a grade de horários institucional. Como não importam as planilhas de horários, a ocupação das aulas teria de ser cadastrada manualmente a cada semestre, exatamente o retrabalho que motiva esta proposta. Além disso, são soluções proprietárias, pagas por assinatura e hospedadas em servidores externos, o que gera custos recorrentes e reduz o controle da instituição sobre os próprios dados.

2.2.2 GESTÃO DE LABORATÓRIOS NO IFNMG CAMPUS ALMENARA

Cruz e Barroso (2023) conduziram uma pesquisa de campo no IFNMG Campus Almenara com o objetivo de analisar a utilização dos espaços e equipamentos dos laboratórios educacionais daquela unidade. O estudo adotou abordagem qualitativa e quantitativa, com coleta de dados por meio de questionários aplicados a servidores e docentes do campus. Os resultados revelaram que a gestão dos laboratórios ocorre de forma informal, baseada exclusivamente em reservas pontuais feitas com antecedência, sem planejamento oficial

permanente e sem qualquer mecanismo para medir o desempenho ou a eficiência no uso desses espaços.

O trabalho de Cruz e Barroso (2023) contribui para a presente pesquisa ao documentar, em um campus do próprio IFNMG, a mesma ausência de ferramentas de gestão identificada no Campus Salinas. A semelhança entre os dois contextos reforça que o problema não é localizado e que a carência de soluções tecnológicas adequadas é uma realidade institucional mais ampla. No entanto, o trabalho desses autores se limita ao diagnóstico: não propõe nem desenvolve qualquer solução de software para o problema identificado. O presente trabalho avança nessa direção ao propor um sistema que automatiza o controle de disponibilidade e registra o histórico de uso dos espaços.

2.2.3 SISTEMA WEB DE RESERVAS NA UESPI – PARNAÍBA

Nascimento, Rios e Baluz (2020) desenvolveram um sistema web para controlar pedidos de reservas de recursos físicos na Universidade Estadual do Piauí, Campus Parnaíba. O contexto era semelhante ao do presente trabalho: as reservas de espaços e equipamentos eram registradas manualmente por servidores em uma agenda digital, e a demanda tornava o processo oneroso e sujeito a falhas. A solução desenvolvida pelos autores automatizou o controle das solicitações, disponibilizou a interface para os usuários da instituição e obteve avaliação positiva quanto à satisfação dos usuários com o sistema.

O trabalho de Nascimento, Rios e Baluz (2020) demonstra que o desenvolvimento de sistemas web de reserva em instituições de ensino é viável e resulta em ganhos concretos para a gestão. A presente proposta compartilha o mesmo problema central, mas apresenta características que a diferenciam. A principal delas é a importação das planilhas de horários institucionais, com reimportação ou ajuste pontual ao longo do semestre. Além disso, oferece consulta pública da ocupação, sem autenticação, e relatórios de uso dos espaços, recursos não descritos no trabalho referenciado. A Tabela 1 resume essa comparação.

2.3 FUNDAMENTAÇÃO TECNOLÓGICA

Esta seção apresenta os conceitos técnicos que embasam o sistema proposto: engenharia de software, UML, banco de dados, controle de acesso, arquitetura web, o *framework* Django, usabilidade e testes.

2.3.1 ENGENHARIA DE SOFTWARE

Sommerville (2018) define software como programas de computador e documentação associada, que pode ser desenvolvido para um cliente específico ou para o mercado em geral. A engenharia de software é a disciplina que se ocupa de todos os aspectos da produção de software, desde a especificação inicial do sistema até as necessidades de manutenção após

Tabela 1 – Comparação entre o sistema proposto e os trabalhos relacionados

Funcionalidade	**Skedda**	**MS Bookings**	**Nascimento, Rios e Baluz (2020)**	**Este trabalho**
Solicitação de reserva pelo próprio usuário	✓	✓	✓	✓
Aprovação das solicitações	✓	P	✓	✓
Notificações automáticas	✓	✓	✓	✓
Consulta pública, sem login	✓	P	×	✓
Importação da grade de horários institucional	×	×	×	✓
Relatórios de uso dos espaços	✓	✓	–	✓
Sem custo de assinatura	×	×	✓	✓

Legenda: ✓ atende; P atende parcialmente; × não atende; – não informado. Cruz e Barroso (2023) não constam por se tratar de estudo diagnóstico, sem sistema desenvolvido.
Fonte: Elaborado pelo autor (2026).

a sua implantação. Deste campo, este trabalho se apoia diretamente em dois pontos: a especificação de requisitos e os atributos de qualidade que orientam as decisões de projeto.

Um conceito central dessa disciplina é a especificação de requisitos. Sommerville (2018) distingue os requisitos funcionais dos não funcionais: os primeiros descrevem as funções que o sistema deve executar e como deve se comportar em situações específicas; os segundos estabelecem restrições ao sistema, como desempenho, segurança e disponibilidade.

Sommerville (2018) também descreve atributos que um sistema de software profissional precisa apresentar para ser considerado de boa qualidade: manutenibilidade, ou seja, a capacidade de evoluir para atender a necessidades futuras; confiabilidade e segurança, que inclui comportamento correto e proteção contra falhas e acessos indevidos; eficiência, que diz respeito ao uso criterioso dos recursos de hardware; e aceitabilidade, que significa ser compreensível e compatível com o que os usuários esperam.

2.3.2 MODELAGEM DE SISTEMAS COM UML

A *Unified Modeling Language* (UML) é uma linguagem de modelagem padronizada para especificar, visualizar e documentar sistemas de software orientados a objetos (BEZERRA, 2007). Bezerra (2007) esclarece que a UML não é uma metodologia de desenvolvimento, mas uma notação que pode ser adotada em conjunto com diferentes processos. Ela oferece um conjunto de diagramas que representam aspectos distintos do sistema, como a sua estrutura estática e o seu comportamento dinâmico.

O diagrama de casos de uso descreve as funcionalidades do sistema do ponto de vista de quem interage com ele. Seus elementos são os atores, que representam papéis desempenhados por usuários ou sistemas externos, e os casos de uso, que representam as funções oferecidas. Um ator pode especializar outro por generalização, herdando suas

interações. Entre casos de uso, a relação «include» indica um comportamento sempre incorporado ao caso base, e a relação «extend», um comportamento opcional que o estende (BEZERRA, 2007).

O diagrama de classes representa a estrutura estática do sistema. Cada classe reúne atributos e operações, e as associações entre classes indicam, pela multiplicidade, quantos objetos de uma se relacionam com os da outra. A composição é uma associação em que a parte não existe sem o todo (LARMAN, 2007).

O diagrama de sequência representa o comportamento dinâmico, mostrando as mensagens trocadas entre os participantes ao longo do tempo. Cada participante tem uma linha de vida vertical, as mensagens aparecem de cima para baixo na ordem em que ocorrem, e o fragmento *alt* representa caminhos alternativos condicionados a uma guarda (LARMAN, 2007).

Larman (2007) destaca que a modelagem com UML permite registrar e comunicar decisões de projeto de forma visual antes de qualquer implementação, o que facilita o alinhamento entre desenvolvedores e demais partes envolvidas.

2.3.3 BANCO DE DADOS RELACIONAL

Um banco de dados é uma coleção organizada de dados inter-relacionados, gerenciada por um sistema que permite armazená-los, recuperá-los e manipulá-los com eficiência (HEUSER, 2009). Heuser (2009) apresenta o modelo entidade-relacionamento como a principal ferramenta para o projeto conceitual de bancos de dados, por permitir representar as entidades do domínio e os vínculos entre elas antes de definir como os dados serão fisicamente armazenados.

O modelo relacional organiza os dados em tabelas formadas por linhas e colunas, onde cada tabela representa uma entidade e cada linha, um registro dessa entidade. A integridade dos dados é assegurada por restrições como chaves primárias e estrangeiras, que garantem a unicidade dos registros e os vínculos entre tabelas (SILBERSCHATZ; KORTH; SUDARSHAN, 2012). Heuser (2009) também descreve o processo de normalização, pelo qual o esquema relacional é reorganizado para eliminar redundâncias e anomalias de atualização. Um banco de dados normalizado garante que cada informação seja armazenada em um único lugar, reduzindo o risco de inconsistências quando os dados precisam ser modificados.

Silberschatz, Korth e Sudarshan (2012) definem um sistema gerenciador de banco de dados (SGBD) como um conjunto de programas que permite criar, manter e acessar um banco de dados. Os autores também descrevem a linguagem SQL (*Structured Query Language*) como o padrão para definir esquemas, inserir dados e realizar consultas em bancos de dados relacionais. A SQL combina uma linguagem de definição de dados,

para criar e alterar tabelas e restrições, com uma linguagem de manipulação de dados, para inserir, atualizar, remover e consultar registros. Para o sistema proposto, o uso de um SGBD relacional é adequado porque os dados a serem gerenciados (espaços físicos, horários, usuários, solicitações de reserva) têm estrutura bem definida e mantêm entre si relacionamentos que o modelo relacional representa com precisão.

Entre os SGBDs relacionais disponíveis, o PostgreSQL é um sistema de código aberto com mais de três décadas de desenvolvimento, reconhecido pela conformidade com o padrão SQL e pela robustez no tratamento de transações. Ele garante as propriedades ACID (atomicidade, consistência, isolamento e durabilidade), oferece suporte a tipos de dados avançados, controle de concorrência multiversão (MVCC) e recursos como busca textual e integridade referencial, características que o tornam adequado a aplicações que exigem consistência e confiabilidade no armazenamento dos dados (The PostgreSQL Global Development Group, 2024).

2.3.4 CONTROLE DE ACESSO BASEADO EM PAPÉIS

O controle de acesso define quais usuários têm permissão para executar quais operações em um sistema. Entre os modelos existentes, o *Role-Based Access Control* (RBAC) proposto por Sandhu et al. (1996) é amplamente adotado em sistemas de informação institucionais.

No modelo RBAC, os direitos de acesso são associados a papéis, e os usuários adquirem esses direitos ao serem atribuídos a um ou mais papéis. Sandhu et al. (1996) descrevem que essa abordagem simplifica a administração, pois as permissões são definidas no nível do papel e não para cada usuário individualmente. Quando as responsabilidades de um usuário mudam, basta alterar a sua atribuição de papéis, sem necessidade de revisar cada permissão específica.

Sandhu et al. (1996) definem quatro modelos de referência que compõem a família RBAC. O RBAC0, o modelo base, define os elementos fundamentais: usuários, papéis, permissões e sessões, onde uma sessão corresponde a um período de atividade de um usuário dentro de um subconjunto dos seus papéis. O RBAC1 estende o modelo base com hierarquias de papéis, permitindo que papéis mais abrangentes herdem as permissões de papéis subordinados. O RBAC2 acrescenta restrições ao RBAC0, incluindo o conceito de separação de obrigações, pelo qual um usuário não pode acumular simultaneamente papéis conflitantes. O RBAC3 combina hierarquias e restrições, reunindo as características dos modelos anteriores.

O sistema proposto segue o RBAC0 com restrições típicas do RBAC2, sem hierarquia de papéis. As restrições adotadas são a cardinalidade, pela qual cada usuário tem no máximo um perfil, e a reserva de permissões administrativas, que não podem ser atribuídas a nenhum perfil. No padrão NIST para RBAC, essa configuração corresponde ao *Core*

RBAC com restrição de cardinalidade (FERRAIOLO et al., 2001). Como cada usuário tem um único perfil, a separação de obrigações entre papéis conflitantes não se aplica.

Duas regras do sistema complementam o modelo por papéis. A primeira trata de quem pode atribuir perfis: um usuário só atribui um perfil se já tiver todas as permissões dele, questão estudada pelo RBAC administrativo (SANDHU; BHAMIDIPATI; MUNAWER, 1999). A segunda considera o vínculo do usuário com o registro, e não só o perfil: o docente de uma aula pode cancelar as ocorrências dela, e disciplinas e turmas ficam limitadas ao escopo do docente. Esse tipo de verificação se aproxima do controle por atributos, combinado ao RBAC em modelos híbridos (KUHN; COYNE; WEIL, 2010).

2.3.5 DESENVOLVIMENTO WEB E ARQUITETURA DE SOFTWARE

Um sistema web é uma aplicação de software acessada por meio de um navegador, que utiliza a infraestrutura da internet para a comunicação entre clientes e servidores. Tanenbaum e Wetherall (2011) descrevem a *World Wide Web* como um sistema de documentos interconectados distribuídos pela internet, baseado no protocolo HTTP para a troca de informações. Esse modelo de comunicação é a base sobre a qual sistemas web são construídos.

Do ponto de vista arquitetural, Laudon e Laudon (2014) descrevem que sistemas de informação baseados na web se organizam tipicamente em três camadas: a de apresentação, responsável pela interface com o usuário no navegador; a de aplicação, onde ficam as regras de negócio; e a de dados, onde as informações são armazenadas. Essa separação em camadas facilita a manutenção e permite que cada parte do sistema seja modificada sem afetar diretamente as demais.

Tanenbaum e Wetherall (2011) descrevem o protocolo HTTP como uma especificação sem estado (*stateless*), o que significa que cada requisição do cliente ao servidor é tratada de forma independente, sem que o servidor retenha informações sobre interações anteriores. Essa característica simplifica a arquitetura dos servidores web, mas exige que o estado da aplicação, como a sessão do usuário autenticado, seja gerenciado explicitamente pela camada de aplicação, por meio de mecanismos como *tokens* de autenticação.

Laudon e Laudon (2014) também apontam que sistemas web têm a vantagem de não exigir instalação de software no dispositivo do usuário: basta um navegador com acesso à internet para que o sistema esteja disponível. Isso reduz os custos de implantação, elimina problemas de compatibilidade entre sistemas operacionais e centraliza a manutenção no servidor, onde atualizações passam a valer imediatamente para todos os usuários. No caso do sistema proposto, essa característica é relevante porque os docentes do campus utilizam dispositivos variados e não dependem de nenhuma configuração específica para acessar a plataforma de reservas.

Para o sistema proposto, a arquitetura em três camadas é adequada porque isola a interface utilizada pelos docentes da lógica que controla as reservas e da base de dados onde os registros são mantidos. Na prática, essa separação é implementada por meio de um *framework* web, que fornece a estrutura de código e os mecanismos de comunicação entre as camadas, reduzindo o trabalho de infraestrutura e permitindo que o desenvolvimento se concentre nas regras de negócio da aplicação. O padrão arquitetural adotado pelo *framework* escolhido para este sistema é detalhado na seção a seguir.

2.3.6 PADRÕES ARQUITETURAIS PARA APLICAÇÕES WEB: MVC E MTV

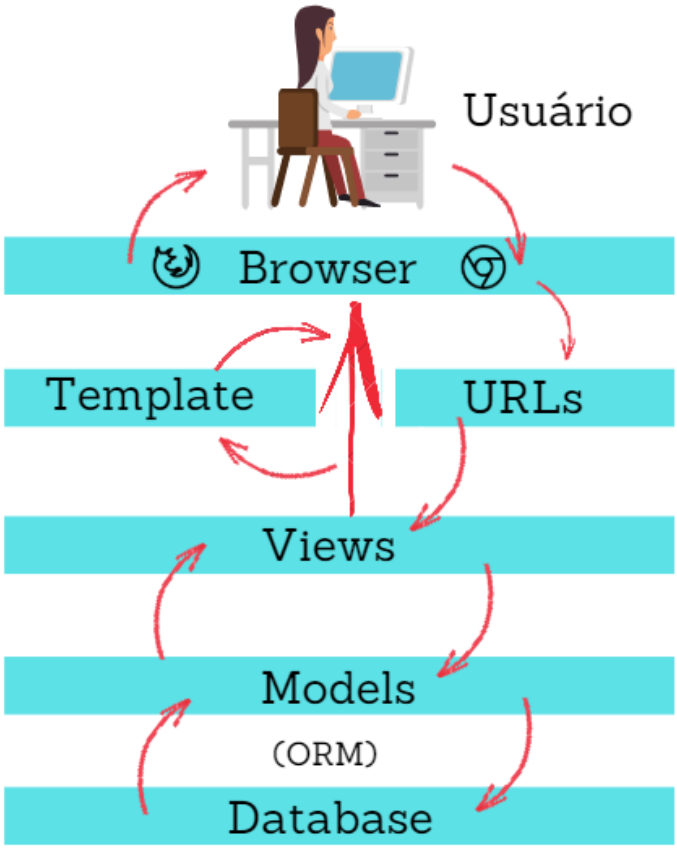
O padrão *Model-View-Controller* (MVC) é uma abordagem arquitetural consolidada para organizar o código de aplicações interativas em três componentes com responsabilidades bem delimitadas. O *Model* gerencia os dados e as regras de negócio da aplicação. A *View* é responsável pela apresentação das informações ao usuário. O *Controller* recebe as ações do usuário, processa as entradas e coordena a interação entre *Model* e *View* (Django Software Foundation, 2024). Essa separação permite que cada componente seja desenvolvido e testado de forma independente.

O *framework* Django adota uma variação desse padrão denominada MTV (*Model-Template-View*), que redistribui as responsabilidades de forma distinta. O *Model* cumpre a mesma função do MVC: representa as entidades de dados e encapsula as regras de negócio associadas ao banco de dados. O *Template* substitui a *View* do MVC e se ocupa exclusivamente da renderização da interface, gerando o HTML entregue ao navegador. A *View* do MTV, por sua vez, assume o papel que no MVC pertenceria ao *Controller*: ela recebe a requisição HTTP, aciona os *Models* necessários e seleciona o *Template* adequado para compor a resposta (TreinaWeb, 2023). A Figura 1 ilustra essa organização.

A diferença mais relevante entre os dois padrões está na forma como o roteamento de requisições é gerenciado. No MVC clássico, o *Controller* é o ponto de entrada das requisições do cliente. No MTV do Django, esse papel é desempenhado pelo próprio mecanismo de roteamento de URLs do *framework*, que mapeia cada endereço recebido para a *View* correspondente por meio do arquivo `urls.py`, antes mesmo de qualquer lógica da aplicação ser acionada (Django Software Foundation, 2024). A Figura 2 compara os dois padrões.

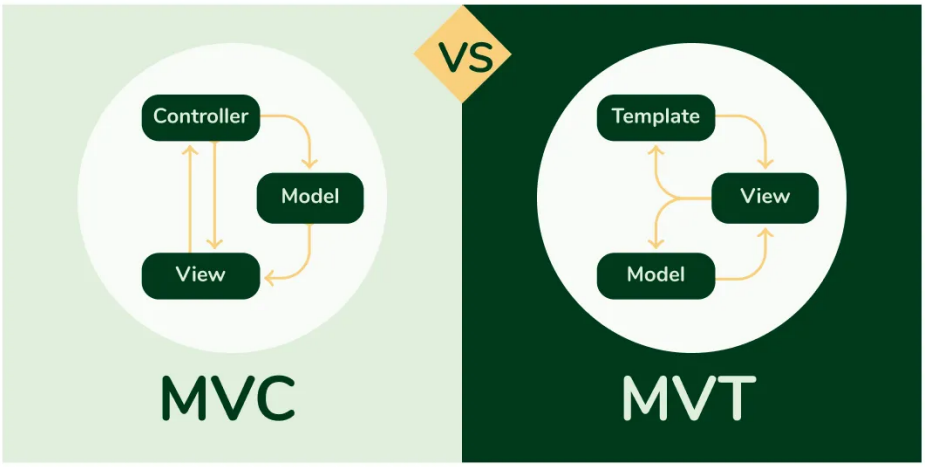
O fluxo de uma requisição no Django, representado na Figura 3, percorre essas camadas de forma sequencial: o cliente envia uma requisição HTTP, o roteador de URLs identifica a *View* responsável, a *View* acessa os *Models* para obter ou persistir dados e, em seguida, seleciona o *Template* adequado para gerar a resposta enviada de volta ao cliente (SILVA, 2023).

Figura 1 – Arquitetura MTV do Django



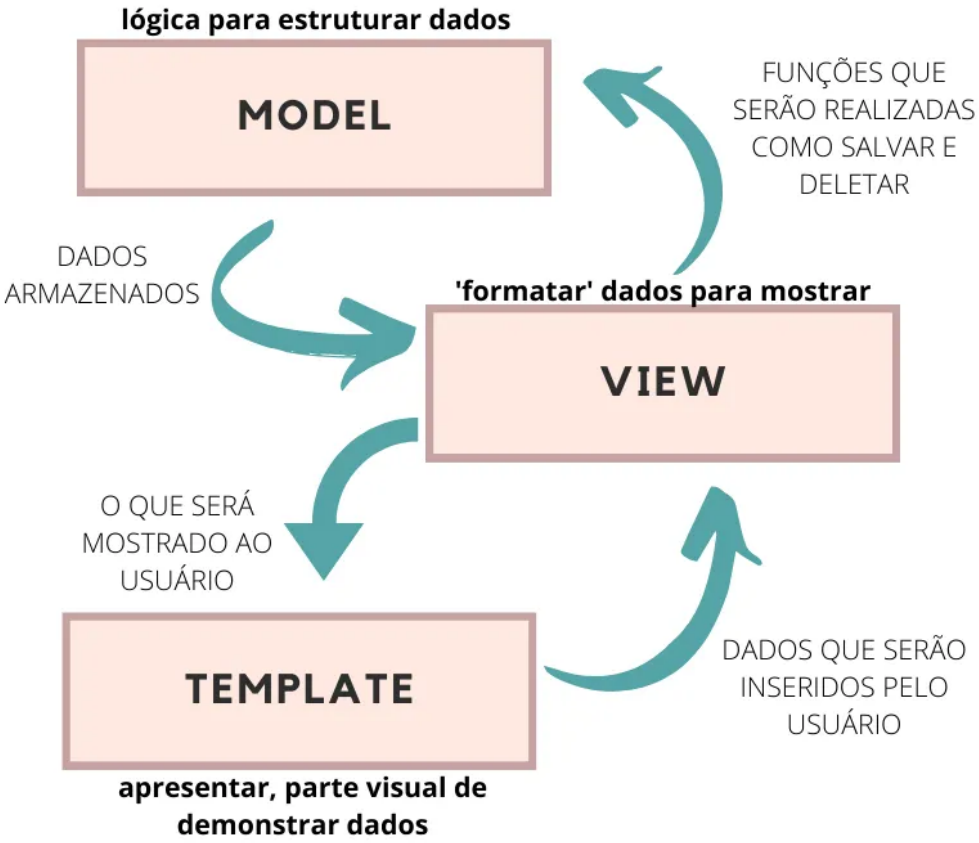
Fonte: (TreinaWeb, 2023)

Figura 2 – Comparação entre as arquiteturas MVC e MTV



Fonte: (Django Software Foundation, 2024)

Figura 3 – Fluxo de requisição na arquitetura MTV do Django



Fonte: (SILVA, 2023)

2.3.7 O FRAMEWORK DJANGO

O Django é um *framework* web de código aberto escrito em Python. Ele opera segundo o princípio *batteries included*: disponibiliza nativamente os componentes mais comuns no desenvolvimento web, como ORM, sistema de autenticação, roteamento de URLs, mecanismo de *templates* e envio de e-mails, sem dependência de bibliotecas externas para essas funções (Django Software Foundation, 2024). Outro princípio central do *framework* é o *Don't Repeat Yourself* (DRY), que orienta o reaproveitamento de código e a centralização de lógicas que, se duplicadas, aumentariam o custo de manutenção ao longo do tempo (Django Software Foundation, 2024).

O roteamento de URLs é configurado no arquivo `urls.py`, que associa cada padrão de endereço a uma *view* específica por meio da função `path()`. Quando uma requisição chega ao servidor, o *framework* percorre a lista de padrões registrados até encontrar o primeiro que corresponda à URL solicitada e delega o processamento à *view* associada.

Esse mecanismo é independente da estrutura de diretórios da aplicação, o que permite organizar as URLs de forma legível e separada da lógica de negócio (Django Software Foundation, 2024).

Cada *model* do Django é uma classe Python que herda de `django.db.models.Model`. Cada atributo da classe representa uma coluna da tabela correspondente no banco de dados, e o tipo do campo, como `CharField`, `DateTimeField` ou `BooleanField`, determina o tipo de dado armazenado e as validações aplicadas (Django Software Foundation, 2024). Relacionamentos entre entidades são representados por campos específicos: `ForeignKey` para associações muitos-para-um e `ManyToManyField` para associações muitos-para-muitos. O ORM traduz as operações sobre esses objetos Python em comandos SQL compatíveis com o banco de dados configurado (Django Software Foundation, 2024).

As mudanças nos *models* são controladas pelo sistema de *migrations*. O comando `makemigrations` gera um arquivo descrevendo as alterações a serem aplicadas no banco; o comando `migrate` as aplica de forma incremental e rastreável, mantendo um histórico completo de todas as versões do esquema (Django Software Foundation, 2024).

O sistema de *templates* do Django utiliza a *Django Template Language* (DTL), que insere dados dinâmicos nas páginas HTML por meio de variáveis e *tags* de controle. A herança de *templates*, obtida com as *tags* `{% extends %}` e `{% block %}`, permite que um *template* base defina a estrutura comum a todas as páginas, como cabeçalho, menu e rodapé, e que os demais *templates* sobrescrevam apenas os blocos de conteúdo específicos (Django Software Foundation, 2024). Esse recurso elimina duplicação de HTML e garante consistência visual em toda a aplicação.

O Django oferece proteções de segurança ativas por padrão. A proteção contra *Cross-Site Request Forgery* (CSRF) é implementada por um *middleware* que exige um *token* válido em todas as requisições que modificam dados no servidor; a *tag* `{% csrf_token %}` insere esse *token* automaticamente nos formulários. A proteção contra *Cross-Site Scripting* (XSS) é garantida pelo sistema de *templates*, que escapa automaticamente todo conteúdo dinâmico antes de inseri-lo no HTML. A proteção contra injeção de SQL é inerente ao ORM, que parametriza internamente as consultas geradas, de modo que dados fornecidos pelo usuário nunca são interpolados diretamente no SQL (Django Software Foundation, 2024).

2.3.8 USABILIDADE E INTERFACE DO USUÁRIO

Usabilidade é a medida em que um produto pode ser usado por pessoas específicas para atingir objetivos específicos com eficácia, eficiência e satisfação num determinado contexto (NIELSEN, 1993). Nielsen (1993) descreve cinco atributos que compõem a usabilidade de um sistema: facilidade de aprendizado, eficiência de uso, facilidade de memorização, baixa taxa de erros e satisfação subjetiva do usuário. Um sistema com

boa usabilidade permite que o usuário realize suas tarefas sem depender de treinamento extensivo nem de suporte constante.

Para orientar o projeto de interfaces, Nielsen (1994) propôs dez heurísticas de usabilidade que se tornaram referência na área: (1) visibilidade do estado do sistema, mantendo o usuário informado sobre o que está acontecendo; (2) correspondência entre o sistema e o mundo real, usando linguagem e convenções familiares ao usuário; (3) controle e liberdade do usuário, permitindo desfazer ações e sair de estados indesejados; (4) consistência e padrões, seguindo convenções da plataforma para que o usuário não precise aprender novos significados; (5) prevenção de erros, projetando para evitar que erros ocorram; (6) reconhecimento em vez de memorização, tornando objetos e opções visíveis para reduzir a carga de memória do usuário; (7) flexibilidade e eficiência de uso, permitindo que usuários experientes realizem tarefas mais rapidamente; (8) estética e design minimalista, evitando informações irrelevantes que competem com o conteúdo principal; (9) ajuda para reconhecer, diagnosticar e recuperar erros, exibindo mensagens claras sobre o que deu errado e como corrigir; e (10) ajuda e documentação, oferecendo suporte quando necessário, de forma fácil de buscar e orientada à tarefa.

Algumas dessas heurísticas têm peso particular no sistema proposto. A visibilidade do estado do sistema é importante porque o docente precisa saber, a qualquer momento, se a sua solicitação de reserva está pendente, aprovada ou rejeitada. A correspondência com o mundo real orienta o uso de termos como “laboratório”, “reserva” e “disponível” em vez de jargões técnicos. A prevenção de erros, por sua vez, inclui mecanismos como a verificação automática de conflitos de horário antes de permitir a solicitação.

No contexto deste trabalho, usabilidade é relevante porque o sistema será utilizado por docentes com familiaridade tecnológica variada. Um sistema que exija treinamento extensivo ou que gere dúvidas frequentes tende a ser abandonado, o que comprometeria o próprio objetivo de substituir o processo manual de agendamento.

2.3.9 TESTES DE SOFTWARE

Testes de software são atividades planejadas com o objetivo de verificar se um sistema se comporta conforme o esperado e identificar defeitos antes que o produto chegue ao usuário final. Sommerville (2018) define teste como um processo de validação e verificação que busca demonstrar que o software atende aos seus requisitos e identificar situações em que ele se comporta de forma incorreta ou inesperada. Pressman e Maxim (2016) complementam essa perspectiva ao afirmar que testar é executar um programa com a intenção explícita de encontrar erros, distinguindo o processo de testes de outras atividades de garantia de qualidade por seu caráter sistematicamente verificativo: um teste bem-sucedido é aquele que revela um defeito até então desconhecido.

Os testes unitários são a forma mais elementar de teste de software e têm como foco

verificar o comportamento de unidades individuais de código de forma isolada, antes de integrá-las ao restante do sistema. Myers, Sandler e Badgett (2011) definem teste unitário como o processo de testar subprogramas, funções ou métodos de maneira independente, de modo que, quando uma unidade falha, a localização do defeito seja imediata, sem a necessidade de investigar interações entre múltiplos componentes. Pressman e Maxim (2016) acrescentam que os testes unitários devem cobrir as estruturas de controle internas de cada componente, as condições de contorno, os caminhos independentes de execução e o tratamento de erros.

No contexto de sistemas web com regras de negócio definidas, os testes unitários têm papel direto na validação da lógica da aplicação. Sommerville (2018) observa que os testes unitários são especialmente eficazes para validar componentes que implementam regras de negócio, pois permitem verificar cada condição individualmente e com precisão. À medida que as unidades validadas são combinadas, os testes de integração verificam se os componentes interagem corretamente entre si (PRESSMAN; MAXIM, 2016).

3 METODOLOGIA

Este capítulo descreve a metodologia da pesquisa. A Seção 3.1 classifica a pesquisa quanto à natureza, aos objetivos e ao procedimento técnico, e a Seção 3.2 descreve as etapas do trabalho.

3.1 TIPO DE PESQUISA

Quanto à natureza, esta pesquisa é aplicada. Sua finalidade é produzir um artefato de software para um problema concreto identificado num contexto institucional real, não ampliar o conhecimento teórico de forma genérica. Gil (2008) distingue a pesquisa aplicada da pesquisa básica precisamente por esse critério: a primeira é motivada pela necessidade de resolver problemas práticos e seus resultados têm aplicação imediata.

Quanto aos objetivos, a pesquisa é descritiva. O ponto de partida é a caracterização do processo atual de agendamento de espaços físicos no IFNMG – Campus Salinas: como ele funciona, onde falha e o que os usuários envolvidos precisam que mude. Gil (2008) define a pesquisa descritiva como aquela que busca descrever as características de determinado fenômeno e estabelecer relações entre suas variáveis, sem que o pesquisador interfira nos fatos observados.

Quanto ao procedimento técnico, a pesquisa adota o estudo de caso em combinação com pesquisa bibliográfica. O estudo de caso é adequado porque a investigação recai sobre um ambiente específico, o Campus Salinas, com histórico, atores e restrições próprios que precisam ser compreendidos antes de qualquer proposta de solução. Yin (2015) define o estudo de caso como uma investigação empírica que examina um fenômeno contemporâneo

em profundidade e em seu contexto real, sendo especialmente indicado quando os limites entre o fenômeno e o contexto não são claramente separáveis. A pesquisa bibliográfica, por sua vez, dá sustentação às decisões de projeto, de modelagem e de escolha tecnológica, com base no que a literatura já consolidou sobre engenharia de software, controle de acesso e desenvolvimento web.

3.2 PROCEDIMENTOS METODOLÓGICOS

Esta pesquisa está organizada em cinco etapas: diagnóstico do problema, levantamento e análise de requisitos, modelagem do sistema, implementação e validação. As etapas foram conduzidas de forma iterativa: a cada reunião com a Direção de Ensino, os requisitos eram refinados e incorporados ao sistema em desenvolvimento. No TCC 1 foram concluídas as quatro primeiras etapas e parte da validação; o TCC 2 compreende a conclusão da validação, a implantação e a análise dos resultados.

3.2.1 DIAGNÓSTICO DO PROBLEMA

A primeira etapa consistiu no diagnóstico do problema. Por meio de conversas semiestruturadas, interações técnicas informais e validações contínuas com a profissional responsável pela gestão dos espaços físicos no campus, foram mapeados o processo atual de agendamento, suas limitações operacionais e as necessidades concretas dos usuários envolvidos, como docentes e equipe administrativa, além da consulta pública, disponível sem autenticação. Essa etapa produziu o insumo necessário para a especificação dos requisitos, cujos resultados são apresentados ao final desta seção.

Os dados desta pesquisa são primários, coletados diretamente junto à fonte por meio de um levantamento iterativo de requisitos, conduzido na forma de conversas semiestruturadas, interações técnicas informais, trocas de mensagens e discussões recorrentes sobre o processo de agendamento, além de validações contínuas da proposta ao longo do desenvolvimento. Gil (2008) aponta a entrevista como uma das técnicas de coleta de dados mais utilizadas nas ciências sociais por permitir que o entrevistador obtenha informações sobre fatos, opiniões, percepções e necessidades de forma direta, com abertura para explorar aspectos não previstos no roteiro inicial. Neste trabalho, essa característica foi estendida para um formato iterativo, em que o diálogo com a fonte se prolongou para além de sessões pontuais.

As interações foram conduzidas com a Diretora de Ensino Técnico do IFNMG – Campus Salinas, que responde pela gestão dos agendamentos de espaços físicos no campus. Como marcos desse processo, foram realizadas três sessões remotas mais estruturadas na fase de especificação: em julho de 2025, novembro de 2025 e abril de 2026. Cada sessão cumpriu objetivos distintos. A primeira, em julho de 2025, teve como foco o mapeamento do processo atual de agendamento: como as reservas são feitas, quem as

gerencia, quais conflitos ocorrem com mais frequência e que informações os usuários precisam acessar. A segunda, em novembro de 2025, foi dedicada à validação dos requisitos preliminares levantados: as informações coletadas inicialmente foram apresentadas de forma estruturada para que a Diretora confirmasse, corrigisse ou complementasse o entendimento do pesquisador. A terceira, em abril de 2026, serviu para confirmar os requisitos finais e verificar aspectos que surgiram durante a modelagem e ainda careciam de esclarecimento. Entre essas sessões, manteve-se interação técnica informal contínua – por mensagens e reuniões breves – para esclarecer dúvidas pontuais sobre o funcionamento do processo e validar decisões de modelagem à medida que surgiam.

Durante o desenvolvimento, foram realizadas mais três reuniões remotas. Em agosto de 2026, a planilha de horários do campus foi apresentada e os requisitos foram refinados. Em setembro de 2026, uma reunião tratou de novos refinamentos, e outra apresentou o sistema em funcionamento à Direção de Ensino.

A opção por esse formato semiestruturado e iterativo se justifica metodologicamente porque o roteiro funcionou como guia, sem impedir que temas relevantes surgidos no decorrer das conversas fossem explorados em profundidade, e porque permitiu que a compreensão do processo se refinasse de forma incremental, acompanhando o avanço da modelagem. Esse formato é mais adequado do que questionários fechados ou entrevistas rigidamente estruturadas quando o objetivo é compreender um processo operacional concreto, com suas particularidades e exceções, cujas nuances só emergem na medida em que a proposta de solução se concretiza.

O diagnóstico mostrou que o sistema em uso exige o cadastro manual de todos os horários ocupados antes de qualquer reserva, e esse cadastro é refeito a cada semestre. Os docentes não consultam a disponibilidade nem solicitam reservas sem passar por um servidor, o que concentra todo o controle em um único ponto. Também não há registro consolidado do uso dos espaços que apoie o planejamento da administração.

Dessas limitações derivam as necessidades que orientaram os requisitos: importar a planilha de horários já produzida pelo campus, marcando automaticamente como ocupados os horários das aulas; manter essa ocupação atualizada por reimportação ou ajuste pontual, sem integração em tempo real com sistemas externos; notificar automaticamente os envolvidos no fluxo de reserva; permitir a consulta pública da ocupação; e gerar um mapa da utilização dos espaços ao longo do semestre.

3.2.2 LEVANTAMENTO E ANÁLISE DE REQUISITOS

A segunda etapa foi o levantamento e a análise de requisitos. Pressman e Maxim (2016) descrevem a engenharia de requisitos como um processo contínuo de identificação, análise, especificação e validação das necessidades dos envolvidos, e ressaltam que a qualidade dessa etapa determina a qualidade do que será desenvolvido depois. A especificação

de requisitos foi elaborada a partir das informações coletadas nas reuniões e do referencial bibliográfico consultado, e organizada em duas categorias: requisitos funcionais e requisitos não funcionais.

Os requisitos funcionais descrevem o que o sistema deve fazer, ou seja, as funções e os serviços que ele oferece aos usuários (SOMMERVILLE, 2018). Como o acesso é definido por perfis configuráveis (Seção 2.3.4), cada funcionalidade fica disponível a quem tiver a permissão correspondente, e não a um perfil fixo. A Tabela 2 apresenta os requisitos funcionais, agrupados nos módulos: ensalamento (RF01, RF02, RF13–RF16); espaços, cursos, disciplinas e turmas (RF03, RF17); usuários e perfis (RF04, RF18–RF20); agenda de reservas (RF05–RF08, RF21–RF23); notificações e comunicados (RF09, RF10, RF24); consulta pública (RF11, RF28); painel, relatórios e auditoria (RF12, RF25, RF26); configurações (RF27); e central de ajuda (RF29).

Tabela 2 – Requisitos funcionais do sistema

Código	**Descrição**
RF01	Importar a planilha de horários institucional do semestre, identificando automaticamente os espaços e os horários ocupados pelas aulas, com prévia do resultado e confirmação antes da gravação
RF02	Atualizar os horários ocupados ao longo do semestre, por reimportação da planilha ou pela manutenção das aulas no próprio sistema (RF13 a RF16)
RF03	Cadastrar e manter os espaços físicos do campus, incluindo os outros nomes pelos quais aparecem na planilha e a desativação de espaços fora de uso
RF04	Gerenciar usuários e atribuir a cada um perfil de acesso
RF05	Consultar a disponibilidade dos espaços físicos
RF06	Submeter solicitações de reserva de espaços físicos
RF07	Aprovar ou rejeitar solicitações de reserva
RF08	Cancelar reservas já aprovadas
RF09	Notificar automaticamente, no próprio sistema, os responsáveis pela análise quando uma nova solicitação de reserva for submetida
RF10	Notificar automaticamente, no próprio sistema, o solicitante sobre o resultado da análise da sua solicitação
RF11	Disponibilizar consulta pública, sem autenticação, aos horários dos espaços por sala, docente, curso ou turma, com o horário original e o calendário por data, sem expor e-mails nem a descrição das reservas
RF12	Gerar relatórios sobre a utilização dos espaços, incluindo o mapa de ocupação por dia e horário e o relatório das solicitações de reserva
RF13	Cadastrar aula recorrente, informando turma, disciplina, docentes, espaço, período de vigência, horário e duração
RF14	Alterar a recorrência de uma aula (dia, horário, duração, espaço ou docentes) a partir de uma data, sem alterar as ocorrências anteriores
RF15	Encerrar a recorrência de uma aula a partir de uma data, preservando o histórico das ocorrências anteriores

Continuação da Tabela 2

Código	Descrição
RF16	Cancelar a ocorrência de uma aula em uma data, sem alterar a recorrência, pelo administrador ou pelo docente da aula
RF17	Cadastrar e manter cursos, disciplinas e turmas
RF18	Criar perfis de acesso e definir as permissões de cada um
RF19	Importar usuários em lote por meio de arquivo CSV
RF20	Enviar ao novo usuário, por e-mail, o link para definir a senha, e permitir a recuperação de senha
RF21	Sugerir ao solicitante uma alternativa à reserva solicitada (outro espaço, data ou horário), que ele pode aceitar ou recusar
RF22	Registrar solicitação de reserva em nome de outro usuário
RF23	Acompanhar as próprias solicitações e ver a própria agenda de aulas e reservas
RF24	Publicar comunicados para todos os usuários, para os usuários de um perfil ou para usuários escolhidos
RF25	Exibir painel administrativo com as solicitações pendentes, a ocupação dos espaços e as pendências da grade (aulas sem espaço ou sem docente, conflitos de docente, choques entre aulas e reservas)
RF26	Registrar em auditoria as ações realizadas no sistema e permitir consultá-las com filtros
RF27	Configurar o servidor de envio de e-mails pela interface do sistema
RF28	Exportar o calendário público do semestre, com os filtros escolhidos, em arquivo iCalendar (.ics), para importação em aplicativos de agenda
RF29	Disponibilizar uma central de ajuda com tutoriais passo a passo de cada funcionalidade, com capturas das telas no computador e no celular, mostrando ao usuário só as funcionalidades liberadas para o seu perfil e abrindo direto na ajuda da tela de onde foi acessada

Os requisitos não funcionais estabelecem as restrições de qualidade que o sistema precisa satisfazer (SOMMERVILLE, 2018). Eles foram definidos a partir dos atributos de qualidade descritos pelo autor: confiabilidade e segurança (RNF01, RNF02, RNF07, RNF08, RNF09); eficiência (RNF03, RNF10, RNF11); aceitabilidade (RNF04, RNF05, RNF13); e manutenibilidade (RNF06, RNF12). Pressman e Maxim (2016) reforçam que esses requisitos condicionam a arquitetura e devem ser considerados desde o início do projeto. A Tabela 3 apresenta os requisitos não funcionais.

3.2.3 MODELAGEM DO SISTEMA

A modelagem utilizou três diagramas da UML: casos de uso, classes e sequência. Os diagramas refletem a versão implementada do sistema.

O diagrama de casos de uso (Figura 4) organiza as funcionalidades em módulos. Como os perfis do sistema são configuráveis (Seção 2.3.4), cada ator representa um conjunto de permissões, e não um perfil fixo: Solicitante, Analista de reservas, Gestor do ensalamento

Tabela 3 – Requisitos não funcionais do sistema

Código	Descrição
RNF01	Controlar o acesso com base em papéis (RBAC), restringindo cada funcionalidade ao perfil correspondente
RNF02	Proteger os dados dos usuários contra acessos indevidos: proteções contra CSRF, XSS e injeção de SQL, senha do servidor de e-mail armazenada cifrada, recuperação de senha que não revela quais e-mails estão cadastrados e encerramento das sessões do usuário quando outra pessoa altera o perfil ou o e-mail dele
RNF03	Ter tempo de resposta adequado nas consultas de disponibilidade, mesmo nos períodos de pico
RNF04	Funcionar no navegador web, sem instalar nada nos dispositivos dos usuários
RNF05	Ter usabilidade compatível com o perfil tecnológico dos docentes e consistência visual entre as telas, sem exigir treinamento extenso
RNF06	Ser manutenível, com separação clara entre as camadas de dados, lógica e apresentação
RNF07	Funcionar sem depender da infraestrutura de horários do campus, sem consultas em tempo real a sistemas externos
RNF08	Impedir reservas sobrepostas de um mesmo espaço, mesmo quando solicitações são analisadas ao mesmo tempo, por restrição de integridade no banco
RNF09	Não permitir alterar nem excluir os registros de auditoria
RNF10	Enviar os e-mails em segundo plano, com novas tentativas em caso de falha, sem atrasar a resposta ao usuário
RNF11	Exibir as notificações em tempo real, sem recarregar a página
RNF12	Ser distribuído em contêineres, para facilitar a implantação em ambientes diferentes
RNF13	Adaptar as telas a computadores e celulares

e Administrador especializam o Usuário autenticado, e o Superadministrador especializa o Administrador. O Visitante representa a consulta pública, sem autenticação.

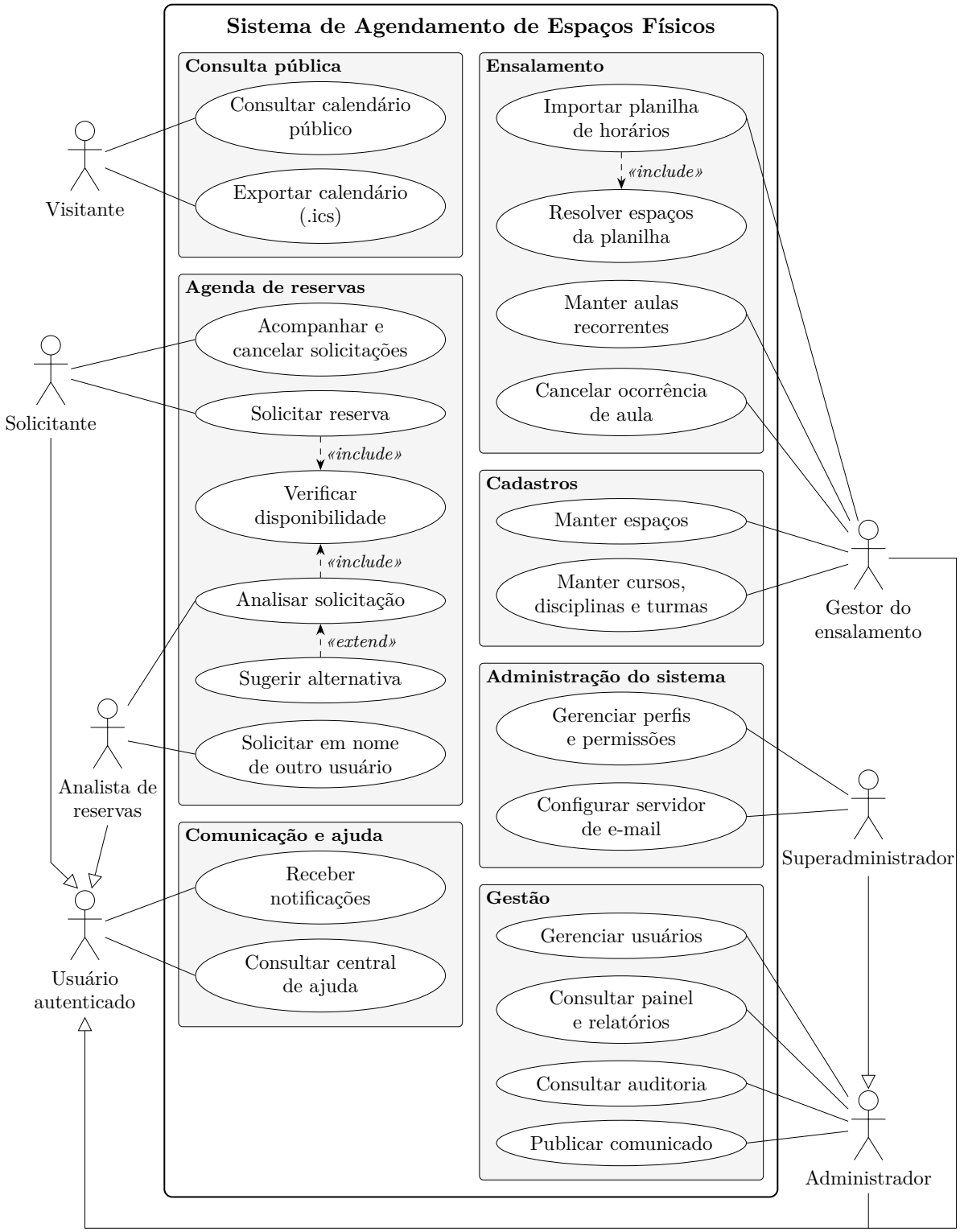
O diagrama de classes (Figura 5) apresenta as 18 classes do domínio, com seus atributos principais e as multiplicidades das associações. Classes de apoio, como importação, auditoria e fila de e-mails, foram omitidas para preservar a legibilidade.

O diagrama de sequência (Figura 6) detalha a solicitação e a aprovação de uma reserva. Os fragmentos *alt* mostram as duas verificações de conflito de horário: a primeira na solicitação e a segunda na aprovação, quando a restrição do banco de dados impede agendamentos sobrepostos (RNF08).

3.2.4 IMPLEMENTAÇÃO

A quarta etapa foi a implementação. O sistema foi desenvolvido como uma aplicação web, com as tecnologias e a arquitetura descritas a seguir, e encontra-se em versão funcional,

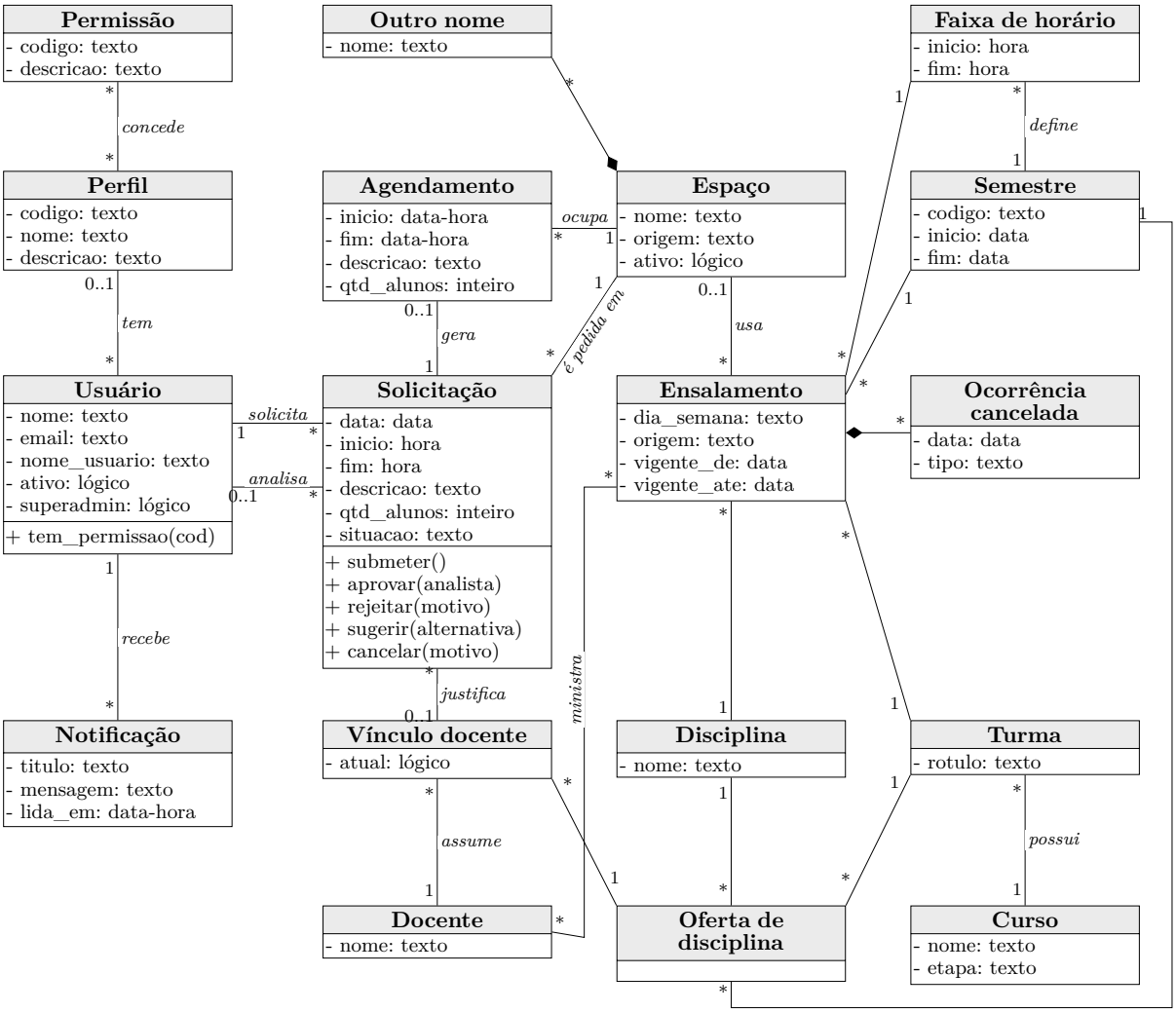
Figura 4 – Diagrama de Casos de Uso do Sistema de Agendamento



Fonte: Elaborado pelo autor (2026).

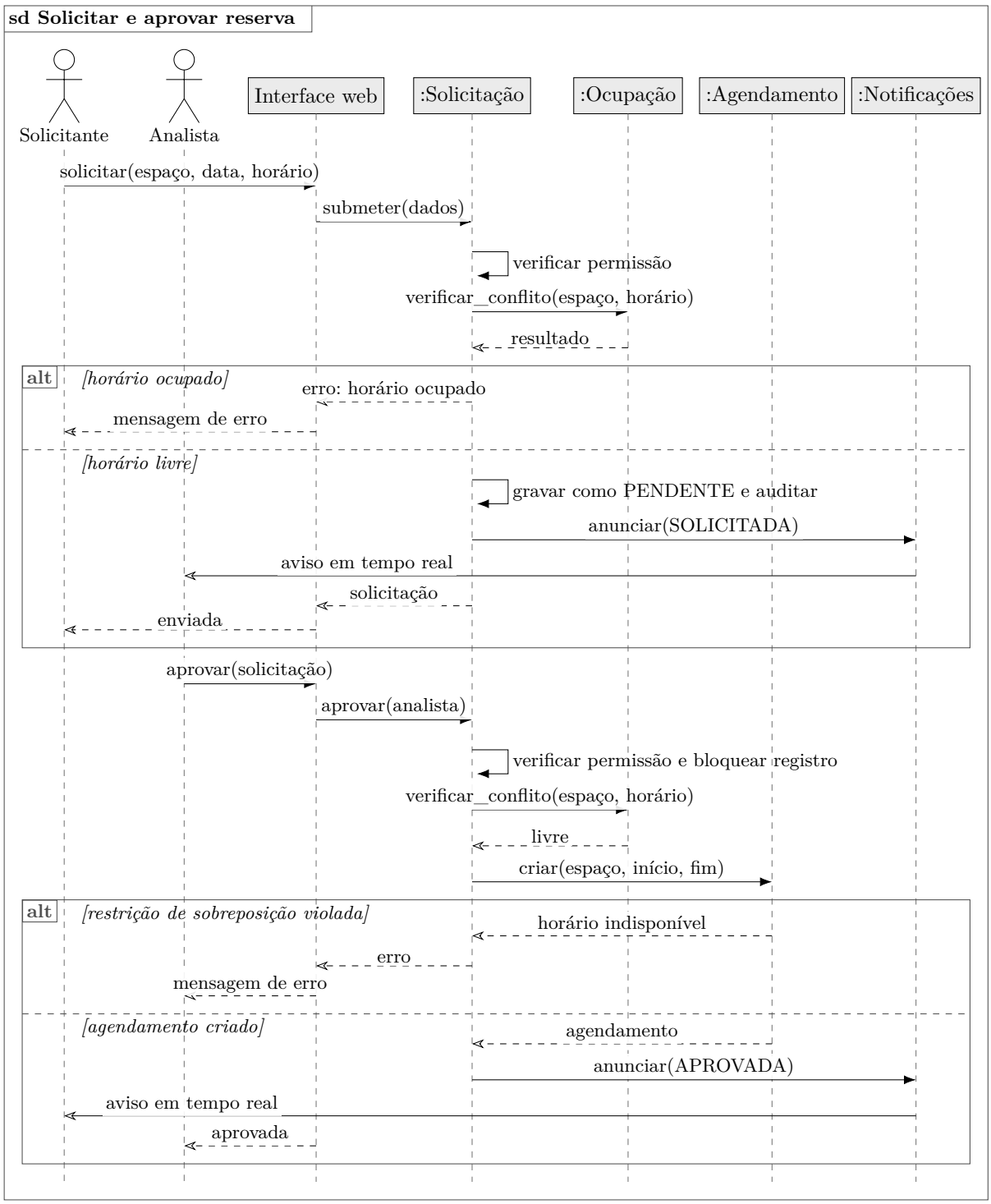
executada em ambiente local e distribuída em contêineres.

Figura 5 – Diagrama de classes do sistema



Fonte: Elaborado pelo autor (2026).

Figura 6 – Diagrama de sequência da solicitação e aprovação de reserva



Fonte: Elaborado pelo autor (2026).

A linguagem de programação adotada é o Python, com o *framework* Django, cujo padrão arquitetural MTV e funcionamento estão descritos nas Seções 2.3.6 e 2.3.7. A escolha se justifica por reunir nativamente os componentes necessários ao sistema, o que reduz a dependência de bibliotecas externas e permite que o desenvolvimento se concentre nas regras de negócio, e pela separação entre dados, lógica e apresentação, que facilita a manutenção do código.

A Tabela 4 resume as funcionalidades do sistema, agrupadas pelos mesmos módulos dos requisitos funcionais.

Tabela 4 – Funcionalidades do sistema por módulo

Módulo	**Funcionalidades principais**
Ensalamento	• Importação da planilha de horários, com prévia e confirmação • Associação dos espaços da planilha aos já cadastrados • Aulas recorrentes: alteração, encerramento e cancelamento de ocorrências
Espaços, cursos, disciplinas e turmas	• Cadastro e manutenção
Usuários e perfis	• Cadastro individual ou em lote (CSV) • Definição e recuperação de senha • Perfis com permissões definidas pelo superadministrador
Agenda de reservas	• Solicitação, inclusive em nome de outro usuário • Análise: aprovação, rejeição ou sugestão de alternativa • Acompanhamento das próprias solicitações
Notificações e comunicados	• Avisos em tempo real • Comunicados para todos, para um perfil ou para usuários escolhidos
Consulta pública	• Calendário por data, com filtros por sala, docente, curso e turma • Horário original da planilha • Exportação do calendário (.ics)
Painel, relatórios e auditoria	• Painel de pendências e solicitações • Mapa de ocupação por dia e horário • Relatório de solicitações • Registro das ações, sem alteração ou exclusão
Configurações	• Servidor de envio de e-mails configurado pela interface
Central de ajuda	• Tutoriais por tela, filtrados pelas permissões do usuário

Para a camada de dados, foi utilizado o PostgreSQL como sistema gerenciador de banco de dados relacional. A escolha se justifica frente a alternativas como o MySQL e o SQLite: embora o SQLite seja prático para ambientes de desenvolvimento, não é recomendado para aplicações com múltiplos acessos simultâneos em produção; o MySQL, por sua vez, é amplamente difundido, mas o PostgreSQL oferece maior conformidade com o padrão SQL, melhor tratamento de integridade referencial e suporte mais completo a transações complexas, recursos relevantes para a consistência das reservas. Soma-se a isso o suporte nativo e robusto do Django ao PostgreSQL, incluindo recursos como busca textual avançada e transações com alto nível de integridade (Django Software Foundation, 2024). O acesso ao banco é feito exclusivamente por meio do ORM, mantendo consistência entre o código Python e o esquema relacional ao longo de todo o ciclo de desenvolvimento.

A camada de apresentação foi construída com os *templates* do Django combinados ao Tailwind CSS. Essa escolha agiliza o desenvolvimento da interface, mantém a consistência visual entre as telas e reduz o custo de manutenção, permitindo ajustes visuais sem alteração nas *views* ou nos *models*.

Para a autenticação dos usuários, foi utilizada a biblioteca *django-allauth*, escolhida por fornecer prontos os fluxos de registro, login, recuperação de senha e gerenciamento de sessões (PENNERSR, 2024). O controle de acesso descrito na Seção 2.3.4 foi implementado com perfis configuráveis. Cada permissão corresponde a uma ação sobre um recurso do sistema, como solicitar uma reserva ou editar um espaço, e o sistema define 47 permissões, distribuídas em 13 módulos. Um perfil é um conjunto dessas permissões, montado pelo superadministrador. Os perfis Administrador e Docente vêm na carga inicial apenas como exemplo e podem ser alterados. Outras categorias, como os técnicos administrativos, recebem um perfil com as permissões de que precisam. A verificação ocorre sempre no servidor, e a consulta pública dispensa autenticação.

As notificações automáticas são exibidas no próprio sistema, em um ícone no cabeçalho que se atualiza em tempo real, sem recarregar a página (RNF11). Ao receber uma solicitação de reserva, o sistema notifica os responsáveis pela análise; após a decisão, o solicitante recebe o resultado (RF09 e RF10). O e-mail é usado apenas para a definição e a recuperação de senha, com o servidor de envio configurado pela interface (RF27) e as mensagens enviadas em segundo plano (RNF10) (Django Software Foundation, 2024).

As notificações em tempo real foram implementadas com o Django Channels, extensão que permite ao Django manter conexões WebSocket com o navegador (Django Software Foundation, 2026). Tarefas demoradas, como a importação da planilha e o envio de e-mails, são executadas por um processo próprio em segundo plano, que lê uma fila registrada no banco e repete o envio em caso de falha. A aplicação, o banco de dados e esse processo são distribuídos em contêineres orquestrados pelo Docker Compose, o que facilita a implantação em ambientes diferentes (Docker Inc., 2026).

Resta definir a infraestrutura de hospedagem do sistema. Segundo a Diretora de Ensino Técnico, está em andamento uma articulação da direção do *campus* com a FADETEC e com a Sifsoft, empresa júnior do curso de Bacharelado em Sistemas de Informação do *campus* Salinas. Como essa articulação ainda não está concretizada, prevê-se, como alternativa, a hospedagem em plataformas gratuitas ou de baixo custo voltadas a aplicações web, a exemplo de Render, Railway ou PythonAnywhere, que oferecem suporte a aplicações Django e permitem disponibilizar o sistema sob um domínio público sem custo de infraestrutura para a instituição.

Cabe esclarecer o escopo deste ponto: o presente trabalho compreende o desenvolvimento e a disponibilização de uma versão funcional do sistema em ambiente acessível por navegador. A implantação definitiva na infraestrutura institucional ficará sob responsabilidade da direção do campus e pelos setores envolvidos.

3.2.5 VALIDAÇÃO

A validação do sistema é feita por testes técnicos automatizados, unitários e de integração, conforme os conceitos da Seção 2.3.9. Os testes são escritos com o *framework* de testes do Django, que executa cada caso sobre um banco de dados temporário (Django Software Foundation, 2024). A versão atual conta com mais de 1.700 casos de teste, que cobrem, entre outras, as regras de negócio críticas listadas na Tabela 5. O critério de aceite é a execução de todos os testes sem falhas. No TCC 2, o sistema passará ainda por homologação com a Direção de Ensino, com dados reais do semestre; testes formais de usabilidade com usuários finais não fazem parte do escopo deste trabalho.

Tabela 5 – Regras de negócio verificadas pelos testes

Regra verificada	**Requisitos**
Bloqueio de reservas sobrepostas, inclusive em análises simultâneas	RF06, RF07, RNF08
Acesso a cada funcionalidade conforme as permissões do perfil	RF18, RNF01
Importação e reimportação da planilha de horários	RF01, RF02
Alteração, encerramento e cancelamento de aulas recorrentes	RF14–RF16
Envio das notificações ao solicitante e aos responsáveis	RF09, RF10
Consulta pública sem exposição de dados pessoais	RF11
Imutabilidade dos registros de auditoria	RF26, RNF09

4 CRONOGRAMA

O cronograma da Tabela 6 abrange o segundo semestre de 2026. As etapas de agosto a outubro, referentes ao desenvolvimento das funcionalidades, já foram concluídas; as de novembro e dezembro correspondem ao TCC 2: conclusão dos testes, implantação, homologação com a Direção de Ensino, escrita e defesa. As atividades foram distribuídas de forma a respeitar a dependência entre as camadas do sistema: a configuração do ambiente e a modelagem do banco de dados precedem o desenvolvimento das funcionalidades, que por sua vez antecedem os testes e os ajustes finais.

Tabela 6 – Cronograma de implementação – 2º semestre de 2026

Etapa	**Meses**				
	Ago	Set	Out	Nov	Dez
Configuração do ambiente e estrutura do projeto	X				
Implementação dos *Models* e banco de dados	X	X			
Desenvolvimento das *Views* e roteamento de URLs		X			
Implementação dos *Templates* e interface		X	X		
Autenticação e controle de acesso (RBAC)		X			
Módulo de importação de planilhas de horários			X		
Módulo de notificações em tempo real			X		
Geração de relatórios de uso			X		
Testes unitários e de integração			X	X	
Ajustes a partir da avaliação do TCC 1				X	
Implantação em ambiente de hospedagem				X	
Homologação com a Direção de Ensino				X	X
Ajustes finais e escrita do TCC 2				X	X
Defesa					X

REFERÊNCIAS

BEZERRA, E. *Princípios de Análise e Projeto de Sistemas com UML*. 2. ed. Rio de Janeiro: Elsevier, 2007. ISBN 9788535216967. Citado 2 vezes nas páginas 6 e 7.

CRUZ, I. N.; BARROSO, D. V. Gestão de laboratórios educacionais: análise da utilização de espaços e equipamentos do IFNMG Campus Almenara/MG. *RECITAL – Revista de Educação, Ciência e Tecnologia de Almenara*, v. 5, n. 1, 2023. Acesso em: abr. 2025. Disponível em: <https://recital.almenara.ifnmg.edu.br/index.php/recital/article/download/270/151>. Citado 5 vezes nas páginas 1, 3, 4, 5 e 6.

Django Software Foundation. *Django Documentation*. 2024. Django Project. Acesso em: abr. 2025. Disponível em: <https://docs.djangoproject.com/>. Citado 6 vezes nas páginas 10, 11, 12, 13, 25 e 26.

Django Software Foundation. *Django Channels Documentation*. 2026. Django Channels. Acesso em: out. 2026. Disponível em: <https://channels.readthedocs.io/>. Citado na página 25.

Docker Inc. *Docker Compose Documentation*. 2026. Docker Docs. Acesso em: out. 2026. Disponível em: <https://docs.docker.com/compose/>. Citado na página 25.

FERRAIOLO, D. F. et al. Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, v. 4, n. 3, p. 224–274, 2001. Citado na página 9.

GIL, A. C. *Métodos e Técnicas de Pesquisa Social*. 6. ed. São Paulo: Atlas, 2008. ISBN 9788522451425. Citado 2 vezes nas páginas 15 e 16.

HEUSER, C. A. *Projeto de Banco de Dados*. 6. ed. Porto Alegre: Bookman, 2009. ISBN 9788577803828. Citado na página 7.

Instituto Federal do Norte de Minas Gerais. *Plano de Desenvolvimento Institucional 2024–2028*. Montes Claros: [s.n.], 2024. IFNMG. Acesso em: abr. 2025. Disponível em: <https://www.ifnmg.edu.br/pdi>. Citado na página 3.

KUHN, D. R.; COYNE, E. J.; WEIL, T. R. Adding attributes to role-based access control. *Computer*, v. 43, n. 6, p. 79–81, 2010. Citado na página 9.

LARMAN, C. *Utilizando UML e Padrões: Uma Introdução à Análise e ao Projeto Orientados a Objetos e ao Desenvolvimento Iterativo*. 3. ed. Porto Alegre: Bookman, 2007. ISBN 9788560031528. Citado na página 7.

LAUDON, K. C.; LAUDON, J. P. *Sistemas de Informação Gerenciais*. 11. ed. São Paulo: Pearson, 2014. ISBN 9788543004792. Citado na página 9.

Microsoft. *Microsoft Bookings*. 2025. Microsoft. Acesso em: 4 jun. 2026. Disponível em: <https://www.microsoft.com/microsoft-365/business/scheduling-and-booking-app>. Citado 2 vezes nas páginas 1 e 4.

MYERS, G. J.; SANDLER, C.; BADGETT, T. *The Art of Software Testing*. 3. ed. Hoboken: Wiley, 2011. ISBN 9781118031964. Citado na página 15.

NASCIMENTO, T. A. d.; RIOS, E. F. S.; BALUZ, R. A. R. S. Sistema Web para Controle dos Pedidos de Reservas de Recursos em Instituições de Ensino Superior. *Revista Eletrônica de Iniciação Científica em Computação*, v. 18, 2020. Acesso em: abr. 2025. Disponível em: <https://journals-sol.sbc.org.br/index.php/reic/article/view/1721>. Citado 3 vezes nas páginas 1, 3 e 5.

NIELSEN, J. *Usability Engineering*. Boston: Academic Press, 1993. ISBN 9780125184069. Citado na página 13.

NIELSEN, J. *10 Usability Heuristics for User Interface Design*. 1994. Nielsen Norman Group. Acesso em: abr. 2025. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Citado na página 14.

PENNERSR, R. *django-allauth Documentation*. 2024. Allauth Project. Acesso em: abr. 2025. Disponível em: <https://docs.allauth.org/>. Citado na página 25.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH, 2016. ISBN 9788580555349. Citado 4 vezes nas páginas 14, 15, 17 e 19.

SANDHU, R.; BHAMIDIPATI, V.; MUNAWER, Q. The ARBAC97 model for role-based administration of roles. *ACM Transactions on Information and System Security*, v. 2, n. 1, p. 105–135, 1999. Citado na página 9.

SANDHU, R. S. et al. Role-based access control models. *IEEE Computer*, v. 29, n. 2, p. 38–47, 1996. Citado na página 8.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 6. ed. Rio de Janeiro: Elsevier, 2012. ISBN 9788535245356. Citado na página 7.

SILVA, D. *Como funciona a arquitetura MTV – Django*. 2023. Medium. Acesso em: abr. 2025. Disponível em: <https://diandrasilva.medium.com/como-funciona-a-arquitetura-mtv-django-86af916f1f63>. Citado 2 vezes nas páginas 10 e 12.

Skedda. *Skedda: Space Booking and Scheduling Software*. 2025. Skedda. Acesso em: 4 jun. 2026. Disponível em: <https://www.skedda.com>. Citado 2 vezes nas páginas 1 e 4.

SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2018. ISBN 9788543024974. Citado 6 vezes nas páginas 5, 6, 14, 15, 18 e 19.

TANENBAUM, A. S.; WETHERALL, D. J. *Redes de Computadores*. 5. ed. São Paulo: Pearson, 2011. ISBN 9788576059240. Citado na página 9.

The PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. The PostgreSQL Global Development Group. Acesso em: 4 jun. 2026. Disponível em: <https://www.postgresql.org/docs/>. Citado na página 8.

TreinaWeb. *Entendendo o MTV do Django*. 2023. TreinaWeb Blog. Acesso em: abr. 2025. Disponível em: <https://www.treinaweb.com.br/blog/entendendo-o-mtv-do-django>. Citado 2 vezes nas páginas 10 e 11.

YIN, R. K. *Estudo de Caso: Planejamento e Métodos*. 5. ed. Porto Alegre: Bookman, 2015. ISBN 9788582602324. Citado na página 15.